

List-2 > count_evens

[prev](#) | [next](#) | [chance](#)

Return the number of even ints in the given array. Note: the % "mod" operator computes the remainder, e.g. 5 % 2 is 1.

count_evens([2, 1, 2, 3, 4]) → 3
count_evens([2, 2, 0]) → 3
count_evens([1, 3, 5]) → 0

Go

...Save, Compile, Run (ctrl-enter)

Show Hint

```
def count_evens(nums):  
    count = 0  
    for n in nums:  
        if n % 2 == 0:  
            count += 1  
    return count
```

Expected	Run		
count_evens([2, 1, 2, 3, 4]) → 3	3	OK	
count_evens([2, 2, 0]) → 3	3	OK	
count_evens([1, 3, 5]) → 0	0	OK	
count_evens([]) → 0	0	OK	
count_evens([11, 9, 0, 1]) → 1	1	OK	
count_evens([2, 11, 9, 0]) → 2	2	OK	
count_evens([2]) → 1	1	OK	
count_evens([2, 5, 12]) → 2	2	OK	
other tests		OK	



All Correct

Good job -- problem solved. You can see our solution as an alternative

List-2 > big_diff

[prev](#) | [next](#) | [chance](#)

Given an array length 1 or more of ints, return the difference between the largest and smallest values in the array. Note: the built-in min(v1, v2) and max(v1, v2) functions return the smaller or larger of two values.

big_diff([10, 3, 5, 6]) → 7

big_diff([7, 2, 10, 9]) → 8

big_diff([2, 10, 7, 2]) → 8

Go

...Save, Compile, Run (ctrl-enter)

```
def big_diff(nums):  
    return max(nums) - min(nums)
```

Expected	Run	
big_diff([10, 3, 5, 6]) → 7	7	OK
big_diff([7, 2, 10, 9]) → 8	8	OK
big_diff([2, 10, 7, 2]) → 8	8	OK
big_diff([2, 10]) → 8	8	OK
big_diff([10, 2]) → 8	8	OK
big_diff([10, 0]) → 10	10	OK
big_diff([2, 3]) → 1	1	OK
big_diff([2, 2]) → 0	0	OK
big_diff([2]) → 0	0	OK
big_diff([5, 1, 6, 1, 9, 9]) → 8	8	OK
big_diff([7, 6, 8, 5]) → 3	3	OK
big_diff([7, 7, 6, 8, 5, 5, 6]) → 3	3	OK
other tests		OK



All Correct

Return the "centered" average of an array of ints, which we'll say is the mean average of the values, except ignoring the largest and smallest values in the array. If there are multiple copies of the smallest value, ignore just one copy, and likewise for the largest value. Use int division to produce the final average. You may assume that the array is length 3 or more.

centered_average([1, 2, 3, 4, 100]) → 3
centered_average([1, 1, 5, 5, 10, 8, 7]) → 5
centered_average([-10, -4, -2, -4, -2, 0]) → -3

Go ...Save, Compile, Run (ctrl-enter)

```
def centered_average(nums):  
    nums.sort()  
    trimmed = nums[1:-1]  
    return sum(trimmed) // len(trimmed)
```

Go

Expected	Run		
centered_average([1, 2, 3, 4, 100]) → 3	3	OK	
centered_average([1, 1, 5, 5, 10, 8, 7]) → 5	5	OK	
centered_average([-10, -4, -2, -4, -2, 0]) → -3	-3	OK	
centered_average([5, 3, 4, 6, 2]) → 4	4	OK	
centered_average([5, 3, 4, 0, 100]) → 4	4	OK	
centered_average([100, 0, 5, 3, 4]) → 4	4	OK	
centered_average([4, 0, 100]) → 4	4	OK	
centered_average([0, 2, 3, 4, 100]) → 3	3	OK	
centered_average([1, 1, 100]) → 1	1	OK	
centered_average([7, 7, 7]) → 7	7	OK	
centered_average([1, 7, 8]) → 7	7	OK	
centered_average([1, 1, 99, 99]) → 50	50	OK	
centered_average([1000, 0, 1, 99]) → 50	50	OK	
centered_average([4, 4, 4, 4, 5]) → 4	4	OK	
centered_average([4, 4, 4, 1, 5]) → 4	4	OK	
centered_average([6, 4, 8, 12, 3]) → 6	6	OK	
other tests		OK	

  All Correct

List-2 > sum13

[prev](#) | [next](#) | [chance](#)

Return the sum of the numbers in the array, returning 0 for an empty array. Except the number 13 is very unlucky, so it does not count and numbers that come immediately after a 13 also do not count.

sum13([1, 2, 2, 1]) → 6

sum13([1, 1]) → 2

sum13([1, 2, 2, 1, 13]) → 6

Go

...Save, Compile, Run (ctrl-enter)

```
def sum13(nums):  
    total = 0  
    i = 0  
    while i < len(nums):  
        if nums[i] == 13:  
            i += 2  
            continue  
        total += nums[i]  
        i += 1  
    return total
```

Expected	Run	
sum13([1, 2, 2, 1]) → 6	6	OK
sum13([1, 1]) → 2	2	OK
sum13([1, 2, 2, 1, 13]) → 6	6	OK
sum13([1, 2, 13, 2, 1, 13]) → 4	4	OK
sum13([13, 1, 2, 13, 2, 1, 13]) → 3	3	OK
sum13([]) → 0	0	OK
sum13([13]) → 0	0	OK
sum13([13, 13]) → 0	0	OK
sum13([13, 0, 13]) → 0	0	OK
sum13([13, 1, 13]) → 0	0	OK
sum13([5, 7, 2]) → 14	14	OK
sum13([5, 13, 2]) → 5	5	OK
sum13([0]) → 0	0	OK
sum13([13, 0]) → 0	0	OK
other tests		OK



All Correct

List-2 > sum67

[prev](#) | [next](#) | [chance](#)

Return the sum of the numbers in the array, except ignore sections of numbers starting with a 6 and extending to the next 7 (every 6 will be followed by at least one 7). Return 0 for no numbers.

sum67([1, 2, 2]) → 5
sum67([1, 2, 2, 6, 99, 99, 7]) → 5
sum67([1, 1, 6, 7, 2]) → 4

Go

...Save, Compile, Run (ctrl-enter)

```
def sum67(nums):
    total = 0
    skip = False
    for n in nums:
        if n == 6:
            skip = True
            continue
        if n == 7 and skip:
            skip = False
            continue
        if not skip:
            total += n
    return total
```

Expected	Run	
sum67([1, 2, 2]) → 5	5	OK
sum67([1, 2, 2, 6, 99, 99, 7]) → 5	5	OK
sum67([1, 1, 6, 7, 2]) → 4	4	OK
sum67([1, 6, 2, 2, 7, 1, 6, 99, 99, 7]) → 2	2	OK
sum67([1, 6, 2, 6, 2, 7, 1, 6, 99, 99, 7]) → 2	2	OK
sum67([2, 7, 6, 2, 6, 7, 2, 7]) → 18	18	OK
sum67([2, 7, 6, 2, 6, 2, 7]) → 9	9	OK
sum67([1, 6, 7, 7]) → 8	8	OK
sum67([6, 7, 1, 6, 7, 7]) → 8	8	OK
sum67([6, 8, 1, 6, 7]) → 0	0	OK
sum67([]) → 0	0	OK
sum67([6, 7, 11]) → 11	11	OK
sum67([11, 6, 7, 11]) → 22	22	OK
sum67([2, 2, 6, 7, 7]) → 11	11	OK
other tests		OK



All Correct

List-2 > has22

[prev](#) | [next](#) | [chance](#)

Given an array of ints, return True if the array contains a 2 next to a 2 somewhere.

has22([1, 2, 2]) → True

has22([1, 2, 1, 2]) → False

has22([2, 1, 2]) → False

Go

...Save, Compile, Run (ctrl-enter)

```
def has22(nums):  
    for i in range(len(nums) - 1):  
        if nums[i] == 2 and nums[i+1] == 2:  
            return True  
    return False
```

Go

Expected	Run		
has22([1, 2, 2]) → True	True	OK	
has22([1, 2, 1, 2]) → False	False	OK	
has22([2, 1, 2]) → False	False	OK	
has22([2, 2, 1, 2]) → True	True	OK	
has22([1, 3, 2]) → False	False	OK	
has22([1, 3, 2, 2]) → True	True	OK	
has22([2, 3, 2, 2]) → True	True	OK	
has22([4, 2, 4, 2, 2, 5]) → True	True	OK	
has22([1, 2]) → False	False	OK	
has22([2, 2]) → True	True	OK	
has22([2]) → False	False	OK	
has22([]) → False	False	OK	
has22([3, 3, 2, 2]) → True	True	OK	
has22([5, 2, 5, 2]) → False	False	OK	
other tests		OK	



All Correct